

Spécifications : Moteur de Recherche Avancé

Projet : **Maryse-Club**

1. Spécifications UX/UI (Pour Claude Design)

L'interface doit rester orientée mobile-first (PWA) tout en offrant une expérience riche sur desktop.

Structure Principale

- **Barre de recherche** : Pré-remplie, avec bouton "Filtres" (ouvre un tiroir sur mobile, barre latérale sur desktop).
- **Résultats** : Grille de RecipeCard. Compteur de résultats. Tri déroulant (Pertinence, Récemment, Mieux noté, Rapide).
- **Pagination** : Bouton "Charger plus" (offset) ou défilement infini. Message d'état vide avec suggestions d'actions.

Filtres à Facettes

- **Classiques** : Type de recette (radio), Difficulté (checkboxes), Temps (curseur), Catégories (tags multiples), Allergènes à exclure (checkboxes).
- **Ingrédients (Inclusion/Exclusion)** : Champ d'autocomplétion. Ajout sous forme de badges (Vert pour inclus, Rouge barré pour exclus) avec un interrupteur bascule lors de l'ajout.
- **Notes** : Sélecteur interactif 5 étoiles pour les recettes. Curseur ou boutons radio pour la note de l'auteur.

2. Spécifications Techniques (Pour Claude Code)

L'objectif est de remplacer le filtrage applicatif actuel par une requête base de données performante et paginée.

Évolution du Schéma de Base de Données

- **Table profiles** : Ajouter `author_rating_avg` (numeric) et `author_rating_count` (int).
- **Table recipes** : Créer une colonne générée `fts` (tsvector) indexée en GIN, agrégeant titre, description, auteur et ingrédients.

Backend / Supabase

- Créer une fonction RPC PostgreSQL `search_advanced_recipes`.
- **Paramètres du RPC** : `search_term`, `filters` (JSONB), `included_ingredients` (text[]), `excluded_ingredients` (text[]), `min_recipe_rating`, `min_author_rating`, `offset_val`, `limit_val`.
- **Logique SQL** :
 - Exclusion ingrédients : `NOT IN (SELECT recipe_id ... WHERE ingredient.name = ANY(excluded))`.

- Inclusion ingrédients : Utiliser l'opérateur @> sur un tableau agrégé des ID d'ingrédients de la recette.

- Mettre à jour `lib/recipes.ts` pour appeler cette RPC.

Frontend (Next.js)

- **État URL** : Sérialiser tous les filtres dans l'URL (ex: ?q=praline&inc=chocolat&exc=fraise&min_rr=4) pour maintenir le rendu serveur (SSR).
- **Composants** : Créer un `SearchFilters` client pour mettre à jour l'URL sans état React lourd. Créer un endpoint d'autocomplétion interrogeant `ingredient_refs`.

Note d'intégration : Les types Supabase (`database.types.ts`) devront être régénérés après la création de la fonction RPC. Le filtrage strict sur les allergènes nécessitera à terme la normalisation de la colonne `ingredients.allergen`.